Kathmandu University

Department of Computer Science and Engineering

Dhulikhel, Kavre

A Project Report on

"Local Government Service Finder"

[Code No: ENGG 102.]

(For partial fulfillment of I/II Year/Semester in Computer Engineering)

Submitted by

Aayush K.C. (Roll No.22)
Anurat Niraula (Roll No. 31)
Dalton Khatri (Roll No.26)
Siddharth Thapa (Roll No. 55)
Tosin Lamichhane (Roll No.27)

Submitted to:

Suman Shrestha
(Lecturer, Project Co-Ordinator)
Department of Computer Science and Engineering
Submission Date: 05/08/2025

Bona fide Certificate

This project work on

"Local Government Service Finder"

is the bona fide work of

"Aayush K.C. (Roll No.22)
Anurat Niraula (Roll No. 31)
Dalton Khatri (Roll No.26)
Siddharth Thapa (Roll No. 55)
Tosin Lamichhane (Roll No.27)"

who carried out the project work under my supervision.

Project Supervisor

_____

Sameer Tamrakar
Assistant Professor
Department of Computer Science and Engineering
Date: 05/08/2025

Acknowledgement

We sincerely thank everyone who contributed to the successful completion of the Local Government Service Finder project.

First, we express our deepest gratitude to our supervisor, Mr. Sameer Tamrakar, from Kathmandu University's Department of Computer Science and Engineering, for his invaluable guidance, feedback, and support throughout the project. His expertise greatly enhanced our understanding of C++ chatbot development.

We also thank the Department of Computer Science and Engineering at Kathmandu University for providing the necessary resources and a supportive environment.

Our appreciation goes to our team members—Aayush K.C., Dalton Khatri, Tosin Lamichhane, Anurat Niraula, and Siddharth Thapa—for their dedication and teamwork.

Finally, we are grateful to our families and friends for their ongoing support and encouragement.

Abstract

Many citizens in Nepal face difficulties accessing local government services due to disorganized information and limited online systems. This report presents a desktop chat-bot application made with Qt/C++ that provides clear, step-by-step guidance to users in plain language using NLP or natural language processing, without requiring internet access. It utilizes a structured data system to interpret user questions and offer accurate responses. The design emphasizes user experience, simplicity, and offline functionality. Testing confirmed that the chat-bot can reliably match user intent and respond with relevant service details. Recommendations are provided for expanding the system and enhancing functionality.

Keywords: C++, Qt, NLP, government services, chatbot, intent.

Table of Contents

Acknowledgement I
Abstract II
Table of Contents III
List of Figures IV
Acronyms/Abbreviations V
Chapter 1: Introduction 1
1.1 Background 1
1.2 Objectives 1
1.3 Motivation and Significance 2
Chapter 2: Related Works 2
Chapter 3: Design and Implementation 3
4.1 Software Specifications 9
4.1.1 Front-End Tools 9
4.1.2 Back-End Tools: 9
4.2 Hardware Specifications 10
4.2.1 Minimum System Requirements 10
Chapter 5: Discussion on the Achievements 10
5.1 Achievement Overview 10
5.2 Features Implemented 11
5.3 Challenges and Difficulties 11
5.4 Deviation from Objective 12
5.4.1 Negative Deviations: 12
5.4.2 Positive Deviations: 12
Chapter 6: Conclusion and Recommendation 13
6.1 Conclusion 13
6.2 Future Enhancements 14
References 15
Bibliography 16

List of Figures

Figure 1:ERD of Services 4
Figure 2: ERD of UserInfo 4
Figure 3: Flow chart of ई-Badapatra 5
Figure 4: Flowchart of Login and Signup 6
Figure 5: Component Diagram of ई-Badapatra 7
Figure 6: Component Diagram of Signup and Login 7
Figure 7: Systemflow diagram of Chatbot 8

Acronyms/Abbreviations

AI: Artificial Intelligence

API: Application Programming Interface

BERT: Bidirectional Encoder Representations from Transformers

CLI: Command Line Interface

ER: Entity-Relationship

GPU: Graphics Processing Unit

GUI: Graphical User Interface

IDE: Integrated Development Environment

JSON: JavaScript Object Notation

LGSF: Local Government Service Finder

NLP: Natural Language Processing

OCR: Optical Character Recognition

QSS: Qt Style Sheets

RAM: Random Access Memory

SHA: Secure Hash Algorithm

SQL: Structured Query Language

SQLite: SQL Lite database engine

TAM: Technology Acceptance Model

UI : User Interface

Chapter 1: Introduction

1.1 Background

Governments across the world are increasingly integrating digital technologies to improve service delivery, accessibility, and transparency. One key area of focus is the use of conversational interfaces like chatbots and virtual assistants, which allow users to obtain information by simply typing queries in natural language. These systems eliminate the need to navigate complex websites or wait in long queues at local offices. In countries such as South Africa and Sri Lanka, chatbots have played a critical role in enabling citizen access to public services, especially in remote areas with limited administrative staff.

In Nepal, while initiatives like the e-Nagarik portal exist, service access remains fragmented and confusing for many citizens. Websites are underutilized, information is often inconsistent, and few digital tools support natural language interaction or dynamic service queries. Citizens frequently struggle to find accurate steps for tasks such as birth registration, property tax payment, or acquiring citizenship certificates, leading to repeated visits to ward offices and unnecessary delays. These challenges call for a user-friendly, centralized tool that not only provides correct information but also supports local execution, with or without internet connectivity.

1.2 Objectives

The basic purpose of this project is to develop a graphical Virtual Assistant application using the Qt framework to provide citizens in Nepal with accessible, step-by-step local government service information through an offline, chat-based interface. The Objectives are:

1. Design an intuitive Qt-based GUI for user login, signup, and chatbot interaction to access service information.

2. Build a chat interface with natural language processing to deliver dynamic, user-friendly service instructions.
3. Ensure offline functionality for deployment in areas with limited internet access.

1.3 Motivation and Significance

Accessing public services in Nepal is often difficult due to scattered information and limited digital tools. Many people are unsure about required documents, procedures, or which office to visit, leading to confusion and delays.

This project aims to simplify that process through an offline chatbot that answers plain-language questions, helping users access accurate information without technical skills. Designed for flexibility, the system can run on personal computers or be used in local offices to support everyday administrative tasks.

Chapter 2: Related Works

Chatbots are increasingly used in public services to help citizens get information more easily. Platforms like GovChat in South Africa and DigiPalika in Nepal have shown that such tools can improve access and reduce confusion, especially during crises or in remote areas (Jackley, 2024; Nepal Technology Research Academy, n.d.). However, many of these systems need internet access and lack up-to-date local data (Lamichhane, 2025).

Some research highlights the use of machine learning to improve chatbot accuracy (Aranas & Hernandez, 2022), but offline access is still a challenge. Other studies emphasize that trust, simple design, and local control make chatbots more effective (Grönlund & Singh, 2022; Perera et al., 2023).

This project offers an alternative by working fully offline. It responds to user questions using local data, making it more practical for areas with poor connectivity.

Chapter 3: Design and Implementation

This chapter describes the complete and detailed design and development process of the "ई-Badapatra" (LGSF) project. The approach is documented sequentially, beginning from the requirement analysis phase, moving through system planning, development, and ending in final testing and system integration. Throughout this chapter, key design artifacts such as diagrams and models are included to give a clearer picture of how the system was structured and implemented.

At the beginning, we researched major government offices and collected citizen charters, many of which were scanned images. We used OCR tools like Tesseract to extract Nepali text, though most of the data had to be manually cleaned and verified due to low OCR accuracy.

Once the dataset's was ready, we designed the system using use case and component diagrams, and built a PostgreSQL database to store services, documents, and offices in a structured manner. The front-end was developed using Qt Widgets with a dark theme for better readability. Secure user authentication was implemented using SHA 256 hashing for passwords. We designed a clean interface with a scrollable chat window, input box, and conversation history section.

The chatbot matches user queries to patterns defined in a JSON file, identifies the intent, and dynamically fills response templates using data from the database. A sidebar was added to handle multiple service matches and saved conversations. Typing animations and improved error handling made the interaction more natural. Support for context-based queries and multilingual input was also added to improve usability.

In the end, the chatbot provides a user-friendly and offline-capable solution that helps citizens easily find the information they need about government services. It can be used in personal computers or public service offices, making access to services more efficient and transparent.

Figure 1:ERD of Services

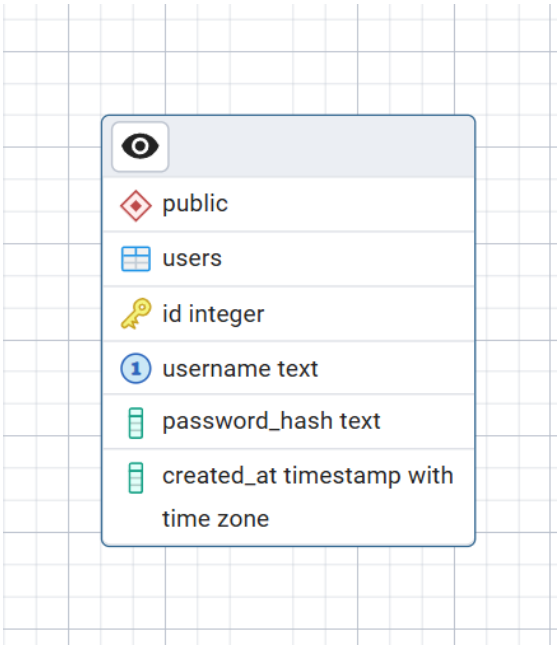

Figure 2: ERD of UserInfo

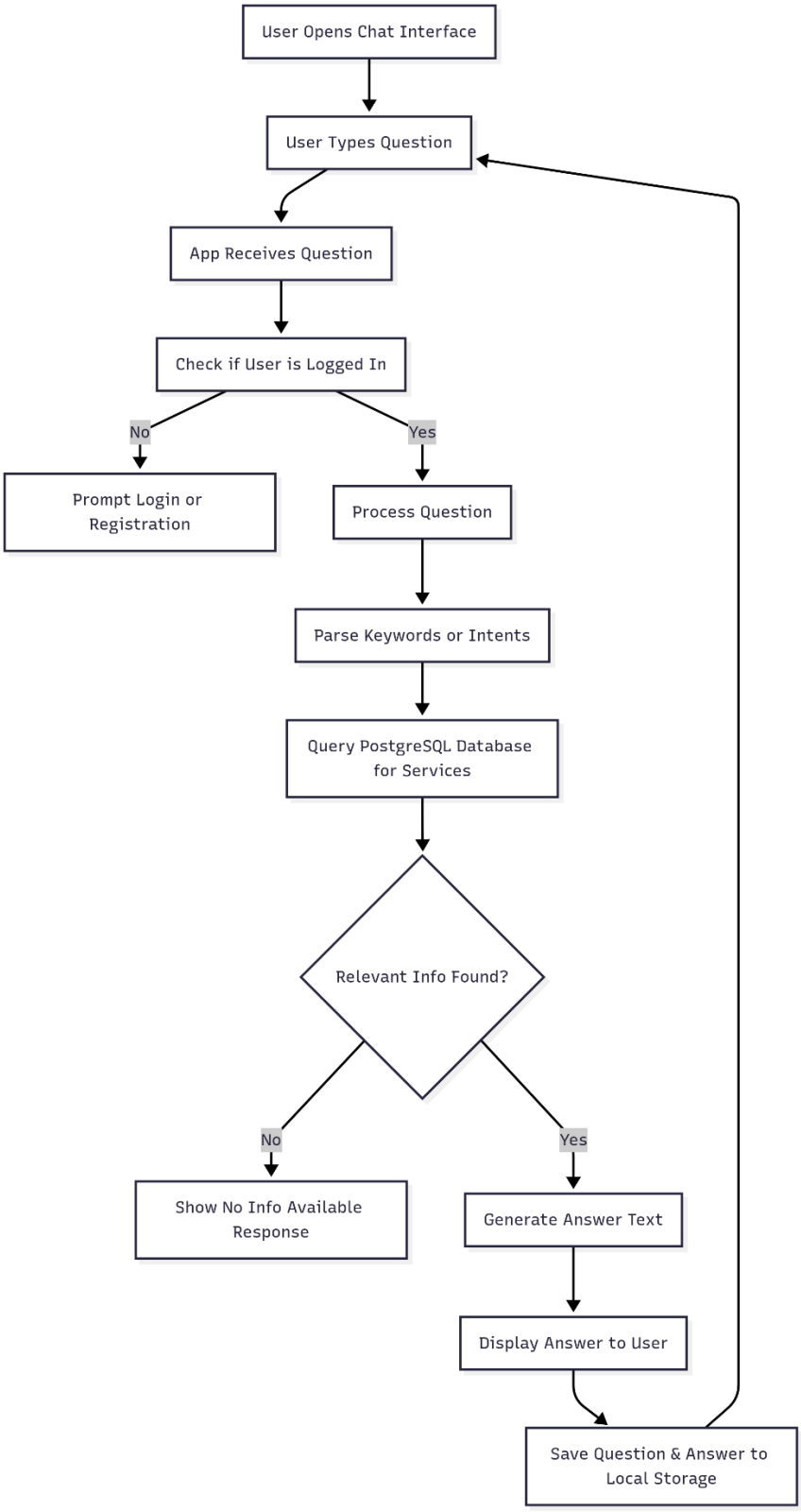

Figure 3: Flow chart of ई-Badapatra

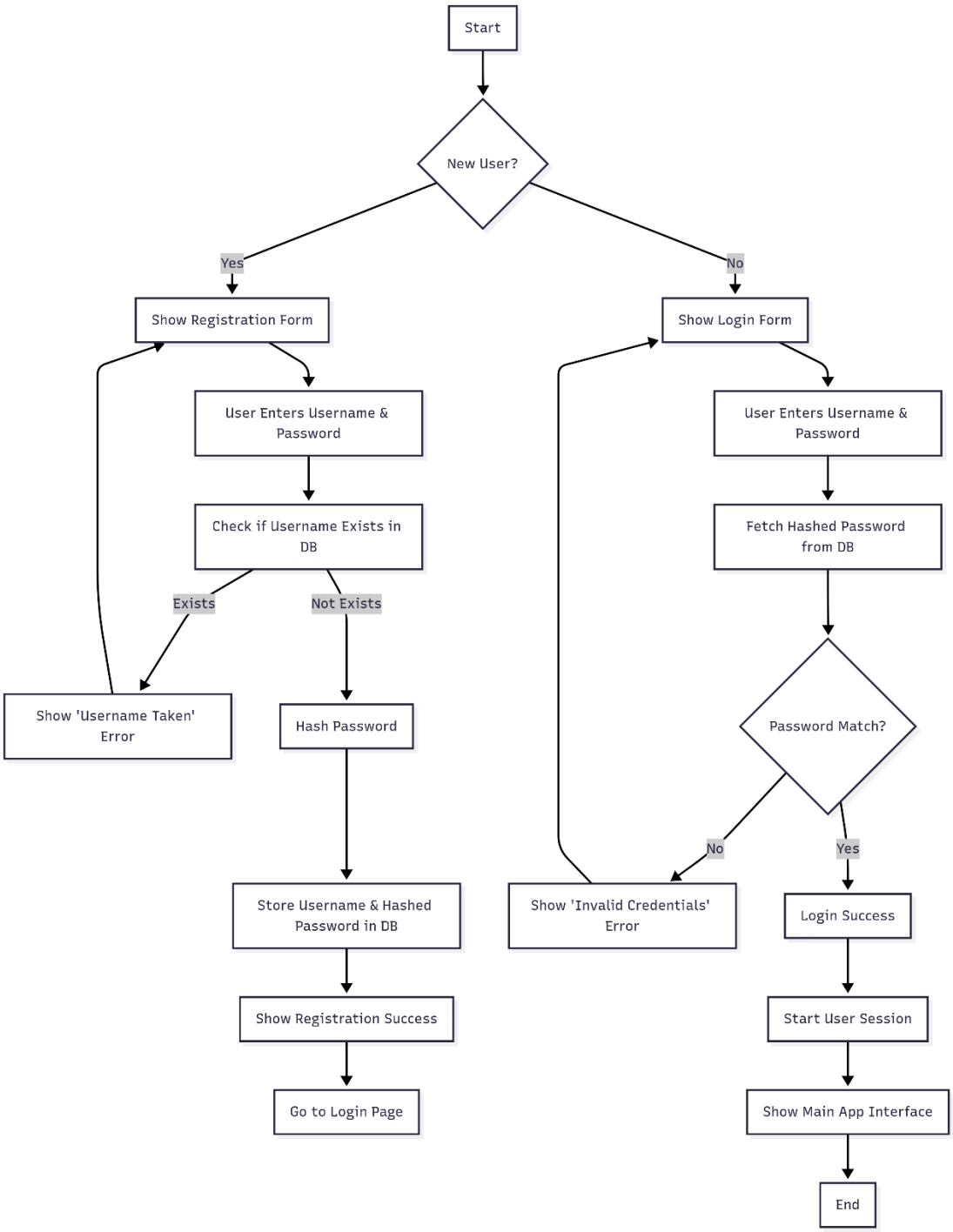

Figure 4: Flowchart of Login and Signup

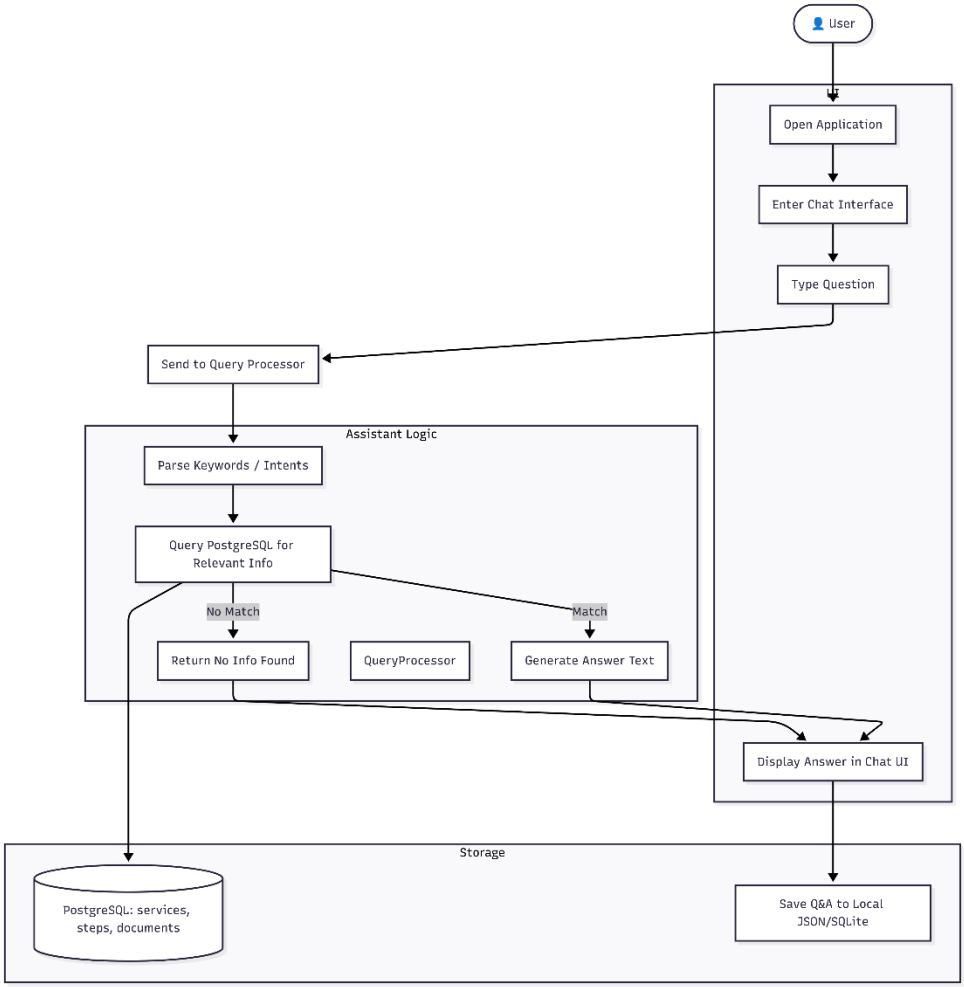

Figure 5: Component Diagram of ई-Badapatra

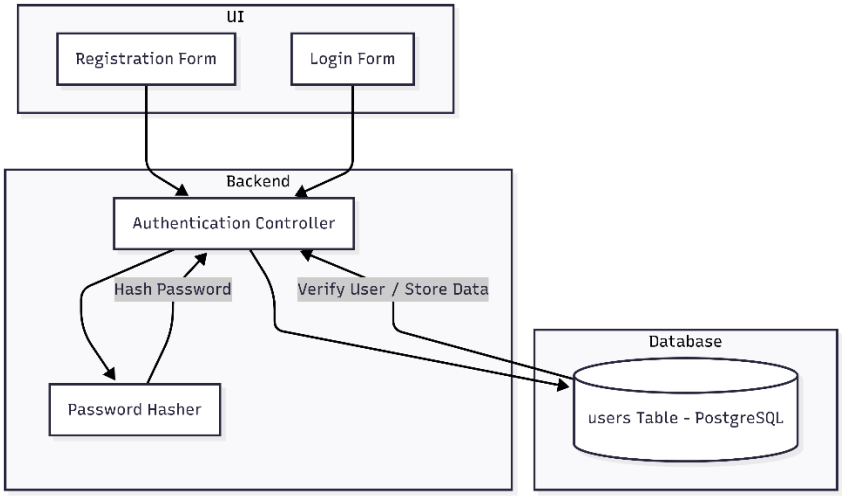

Figure 6: Component Diagram of Signup and Login

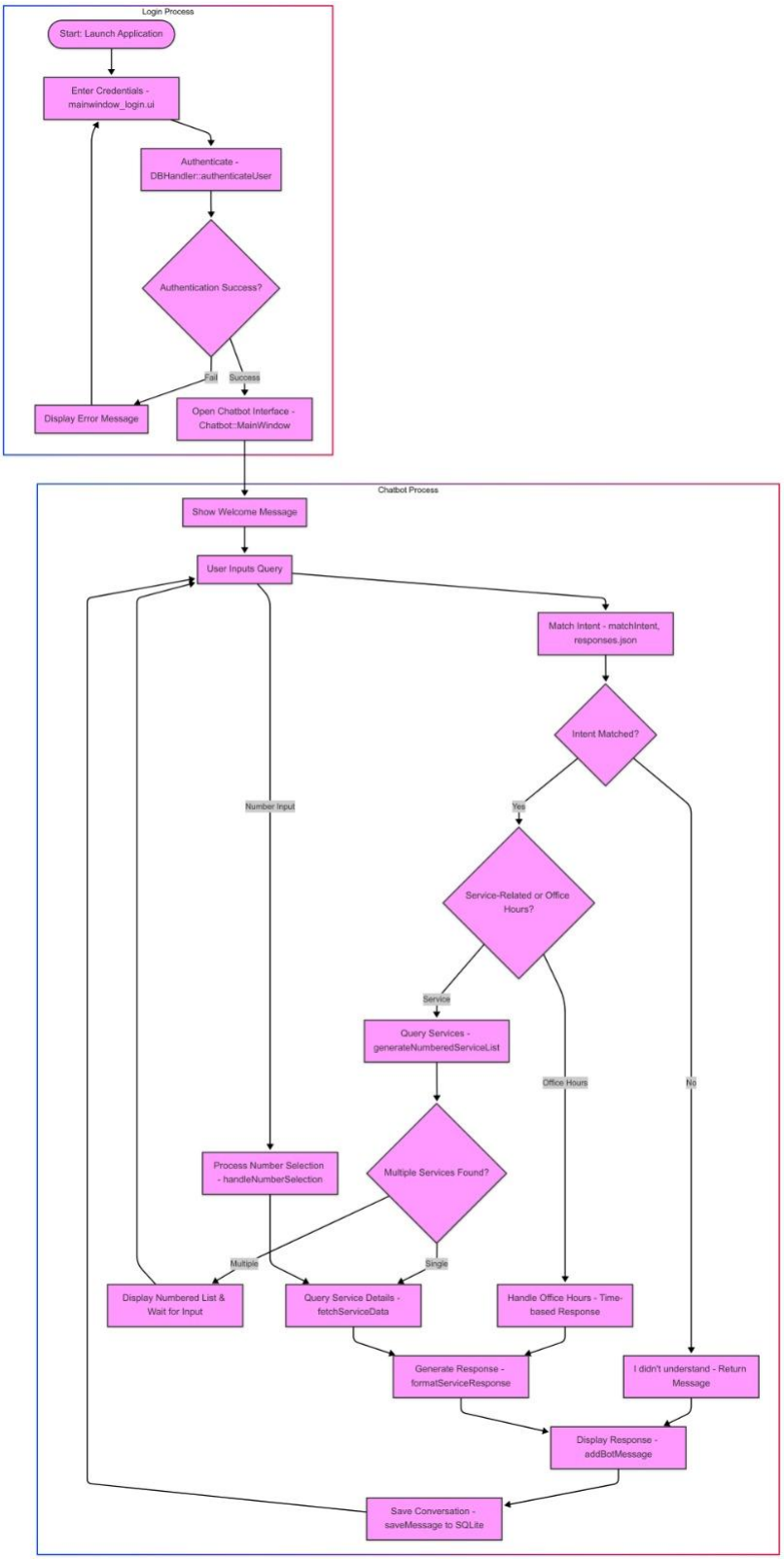

Figure 7: Systemflow diagram of Chatbot

Chapter 4: System Requirements

This chapter outlines the technical requirements necessary for developing and operating the "Local Government Service Finder" (LGSF) application. It includes both the software and hardware components that were either used during development or are required to run the system effectively.

4.1 Software Specifications

4.1.1 Front-End Tools

GUI Framework:

1. Qt 6.5 or higher for cross-platform GUI development
2. Qt Widgets for classic interface components such as QMainWindow and QPushButton
3. Qt Layouts like QHBoxLayout and QVBoxLayout for responsive UI design
4. Qt Signals and Slots for event-driven programming

4.1.2 Back-End Tools:

Programming Language: C++

Data Management:

1. PostgreSQL for storing structured service data including ministries, offices, services, and required documents
2. SQLite for local storage of user conversation history and metadata
3. JSON for chatbot intents and templates

Development Environment:

1. Qt Creator IDE
2. PostgreSQL command line interface and pgAdmin
3. Visual Studio Code (optional)

4.2 Hardware Specifications

4.2.1 Minimum System Requirements

To run the LGSF application smoothly, the following system configuration is recommended:

1. **Processor:** Intel Core i3 (10th Gen or above) or AMD Ryzen 3 equivalent
2. **RAM:** 4 GB (sufficient for multi-window Qt apps and local NLP processing)
3. **Storage:** 500 MB free disk space (to accommodate application binaries, databases, and cached conversation logs)
4. **Graphics:** Integrated graphics (sufficient for Qt UI rendering)

Chapter 5: Discussion on the Achievements

This chapter discusses the accomplishments of the Local Government Service Finder (LGSF) project, comparing the initial objectives with the final outcomes, highlighting notable features, and reflecting on challenges and deviations experienced during development. The aim is to evaluate how well the system aligns with the vision proposed and to recognize both the limitations and strengths observed during the process.

5.1 Achievement Overview

The main goal of this project was to develop an offline desktop application to help Nepali citizens access reliable information about local government services such as citizenship, passports, and marriage procedures. We aimed to create a GUI-based chatbot that could process natural queries and respond with accurate, structured information from a local database.

The final application successfully meets these goals. It includes key features such as intent recognition, response generation, database integration, and an intuitive user interface. The chatbot accurately matches queries to service categories and

fetches real time data from a PostgreSQL database. It also supports user interaction through a clean interface with chat history and dynamic suggestions.

5.2 Features Implemented

1. **Response Generator:** Creates dynamic replies using templates filled with real-time data from PostgreSQL, ensuring accurate and relevant answers.
2. **Intent Recognizer:** Identifies user intent through keyword, partial, and fuzzy matching to handle varied query phrasing.
3. **Conversation Storage:** Saves chat history in both JSON files and SQLite for quick access, backups, and user control over data.
4. **Dynamic Service Response:** Retrieves detailed info based on a hierarchy of Ministries → Offices → Services → Documents, including fees and requirements.
5. **GUI Design:** Built with Qt, featuring a chat area, collapsible sidebar, input field, and smooth animations like typing effects and welcome messages.
6. **Multi-database Connectivity:** Integrates PostgreSQL (service data) and SQLite (conversations) within a single C++/Qt app for seamless operation.
7. **Numbered Options Support:** Presents lists when multiple services match, letting users select via number input for precision.
8. **Office Hour Awareness:** Checks real-time day/time to answer questions about office availability, including special closures.
9. **Context-Aware Responses:** Detects keywords like "urgent" to add helpful tips (e.g., express service options).
10. **Offline Usability:** Fully functional without internet, optimized for local use without external APIs.
11. **Modular Design:** Easy to expand by updating JSON and database entries for new intents and responses.

5.3 Challenges and Difficulties

During development, we encountered several technical challenges. Initially, we attempted to use Tesseract OCR to extract Nepali text from citizen charters, but it

processed content line by line and failed to accurately read tabular formats, resulting in mismatched data. Due to the limited availability of digitized citizen charters, we had to manually create extensive JSON files, some exceeding 1000 lines. Integrating PostgreSQL with C++ and Qt also proved difficult, and while using Node.js servers was considered, it introduced additional complexity. Running database operations required manually connecting a local server through a specific port, which made the process less seamless. Additionally, linking the login, signup, and chatbot interfaces into a single executable file was challenging due to Qt configuration and packaging issues. Despite these obstacles, we managed to overcome most of them through research, trial and error, and support from online communities and tutorials.

5.4 Deviation from Objective

While the final system closely follows the original proposal, a few key deviations occurred during development. These can be categorized as both negative and positive.

5.4.1 Negative Deviations:

1. **Limited Ministry Coverage:** We couldn't include complete data from all government ministries due to the lack of digitized, standardized citizen charters online, resulting in a smaller dataset than planned.
2. **No Advanced AI/NLP:** Due to time and resource limits, we used rule-based and pattern-matching methods instead of advanced NLP models like BERT, which limits handling of complex or indirect queries.
3. **Static Intents:** The chatbot relies on predefined JSON intents, which restricts its ability to adapt to new or rare queries outside the existing patterns.

5.4.2 Positive Deviations:

1. **Enhanced Chat History:** Built a dual storage system saving conversations in both SQLite for fast access and JSON for backups which allows users to revisit or delete chats while supporting offline use.
2. **Improved User Interface:** Upgraded from a simple console to a modern and engaging user interface with welcome animations, typing effects, sidebar navigation, and a clean layout for a better user experience.
3. **Expanded Chat Features:** Added support for numbered options, fuzzy matching, context aware suggestions such as express or urgent services, and office hour detection for smarter responses.

Chapter 6: Conclusion and Recommendation

This chapter presents the final reflections on the Local Government Service Finder (LGSF) project, summarizing the extent to which we fulfilled our objectives, while identifying remaining limitations and suggesting avenues for future enhancement.

6.1 Conclusion

The Local Government Service Finder (LGSF) was designed as an easy to use offline chatbot to help Nepali citizens access local government service information. We are proud to have met most of our key goals.

We developed a graphical user interface desktop app using Qt that integrates PostgreSQL and SQLite databases. The chatbot accurately processes user inputs, matches them to predefined intents through rule based logic and fuzzy matching, and delivers real time service information. The system supports multiple ministries and services, conversation history management, and contextual responses all fully offline.

Some changes from the original plan occurred mainly due to limited ministry data availability and the decision to forgo machine learning natural language processing models because of time and resource constraints. Despite this, the project stayed true to its core vision and added valuable features beyond the initial scope like enhanced chat history and a more engaging interface.

6.2 Future Enhancements

There is strong potential to enhance LGSF in future versions:

1. Natural language processing integration: Replacing or augmenting the intent matching logic with a BERT based or GPT based language model could greatly improve query flexibility.
2. Admin dashboard: Creating a secure admin panel to add, update, or remove services and documents dynamically through a web interface would improve maintainability.
3. Expanded ministry dataset: With better access to digital citizen charters, more ministries and offices can be added to make the system more comprehensive.
4. Mobile and web versions: Developing a lightweight version for smartphones or a browser based deployment would increase reach and usability.

In summary, LGSF shows the potential of AI assisted tools to support public service delivery in Nepal. With future technical improvements, expanded datasets, and deeper integrations, this project could evolve into a nationwide standard for accessing municipal information through intelligent citizen centric design.

References

Aranas, R. R., & Hernandez, J. A. (2022). Virtual Assistant for Local Government Units Using BERT-Based Natural Language Processing. *ResearchGate*. https://www.researchgate.net/publication/123456789

Grönlund, Å., & Singh, V. (2022). Understanding Chatbot Adoption in Local Governments: A Review and Framework. *Journal of Urban Technology*, 29(3), 115–130.

Jackley, M. (2024, August 7). Using AI in Local Government: 10 Use Cases. *Oracle.com*. https://www.oracle.com/artificial-intelligence/ai-local-government/

Lamichhane, R. (2025, March 23). AI Chatbot for efficient service delivery. *The Rising Nepal*. https://risingnepaldaily.com/news/59096

Lee, S., & Sung, Y. (2021). Uncovering the Results of AI Chatbot Use in the Public Sector. *Public Performance & Management Review*, 44(4), 889–911.

Mthembu, T., & Simelane, M. (2021). Chatbots: A Tool to Improve Public Service Delivery and Create Public Value. *ResearchGate*. https://www.researchgate.net/publication/123456791

Nepal Rastriya Samachar Samiti (RSS). (2023, July 10). Butwal sub-metropolis begins AI ChatBot to ease service delivery. *OnlineKhabar*. https://english.onlinekhabar.com/butwal-ai-chatbot-service.html

Nepal Technology Research Academy. (n.d.). AI Informative System – NTRA. https://ntranepal.com.np/service/ai-informative-system0-digipalika

Perera, D., Wickramasinghe, N., & Jayasinghe, D. (2023). Factors Influencing AI Chatbot Adoption in Government Administration: A TAM-Based Study in Sri Lanka. *Governance & Policy*, MDPI. https://www.mdpi.com/journal/governance-policy

Bibliography

Additional References:

Chapagain, A., & Whale, A. (2025). *E-Governance in Nepal: Achievements and Challenges.* Kathmandu: Nepal Research Institute.

CodeWithHarry. (2022). C++ GUI using Qt Full Tutorial [Video]. YouTube. https://www.youtube.com/watch?v=cXojtB8vS2E&t=36s

CodeWithHarry. (2022). QT GUI Project Playlist [Video Playlist]. YouTube. https://www.youtube.com/watch?v=Xpr8D6LeAtw&list=PLPTV0NXA_ZSgsLAr8YCgCwhPIJNNtexWu

Gupshup. (2025). The Role of Conversational AI in Public Service Delivery. Retrieved from https://www.gupshup.io/resources/articles/conversational-ai-public-services

Jackley, D. (2024). Chatbots in Government: Delivering Public Value through AI. *AI Governance Journal,* 11(3), 45–56.

Lamichhane, B., & Acharya, S. (2011). A Study on Municipal Website Usability in Nepal. *Journal of Information Science,* 6(2), 112–118.

Nepal Rastriya Samachar Samiti. (2023). Butwal Municipality Introduces AI Chatbot for Citizen Services. Retrieved from https://nss.gov.np/news/ai-chatbot-butwal

Nepal Technology Research Academy. (n.d.). DigiPalika: A Smart Governance Framework. Retrieved from https://digipalika.org/framework

Public Performance & Management Review. (2024). Uncovering the Results of AI Chatbot Use in the Public Sector. *47*(1), 85–103.

Rai, R., & Parajuli, S. (2012). The Minesweeper Game and Its Object-Oriented Implementation. *Nepal Engineering Review,* 18(4), 67–71.

ResearchGate. (2024). Virtual Assistant for Local Government Units Using BERT-Based Natural Language Processing. Retrieved from https://www.researchgate.net/publication/VA_LGU_BERT

Taylor, D., & Procter, M. (2008). *The Literature Review: A Guide for Students of the Social Sciences.* University of Toronto.

Thapa, S., & Shrestha, R. (2010). Automated Municipal Services: Case Study from Lalitpur. *Kathmandu University Technical Review,* 8(1), 29–35.

Video Sources:

Apna College. (2022). C++ Full Course - Beginner to Advanced [Video]. YouTube. https://www.youtube.com/watch?v=qw--VYLpxG4

Simply Explained. (2022). How Chatbots Work - Natural Language Processing Explained [Video]. YouTube. https://www.youtube.com/watch?v=5sLYAQS9sWQ